

Claude Shannon's implementation of Boolean Algebra (a mathematical system) by George Boole in electronics/circuits.

Date

COMPUTER ORGANIZATION AND ARCHITECTURE:-

Historically there are two types of computers:

↓
COMPUTERS

FIXED PROGRAMS / DEDICATED
SYSTEM OR DEVICE / EMBEDDED
SYSTEM

↓
STORAGE PROGRAM COMPUTERS /
GENERAL PURPOSE COMPUTER /
VON NEUMANN ARCHITECTURE

★ Their function is very specific and they couldn't be reprogrammed e.g. calculators, washing machines etc.

★ They can be programmed to carry out many different tasks; applications are stored on them hence the name.

★ Their functionality is written in terms of program which is permanently fused in a chipset.

★ The idea of a system which can store data in memory, call it on runtime for processing and everything that a modern computer is capable of, is based upon the idea of proposed by Von-Neumann (founder of Merge Sort as well); his concept varied from traditional fixed Program computers.

e.g. microwave

• electric thermometer

Vending Machines.

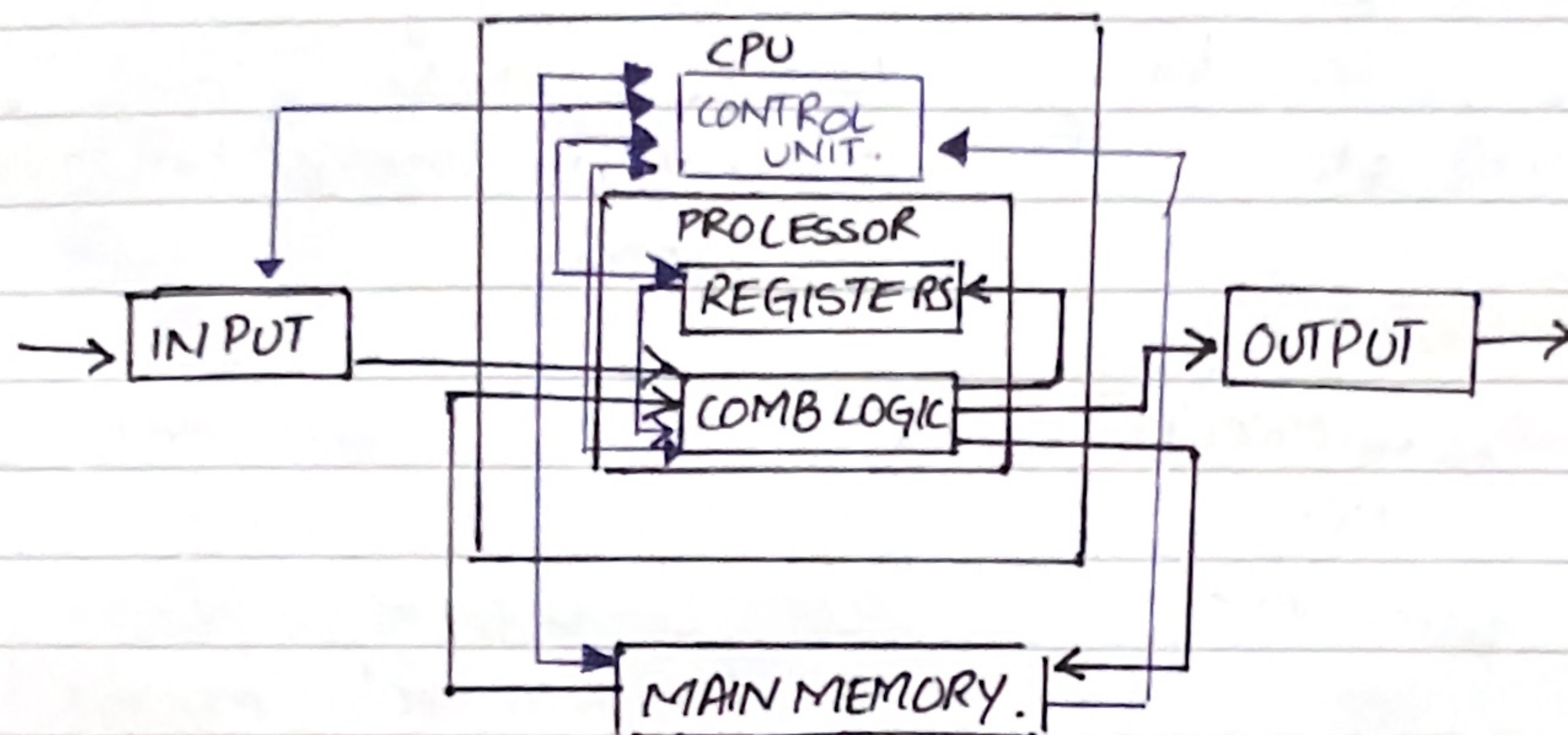
Ventilators.

Date

Stored Program Computers can solve anything which can be theoretically be done by a Turing Machine.

These computers work on a stored program concept where a programmer, writes a program, store it in memory and when computer executes the program it is called back from the memory. This program can also be changed as per the required functionality.

VON - NEUMANN ARCHITECTURE:-



CONTROL UNIT:-

Which generates control signal (master) to control every other part of the CPU. It directs all inputs and output flow, fetches codes for instructions and controls how data, around the system.

COMBINATIONAL
LOGIC = ALU.

(moves)

REGISTERS:-

a few registers that are important are :

- (i) instruction register; contains address of the current register
- (ii) program counter; contains address of the next instruction.
- (iii) Base register; contain base address of program.

Date

ALU:-

a complex combinational circuit which can perform arithmetic operations, Bit shifting operations and logical operations e.g +, - comparison etc.

how in general; operations are performed.

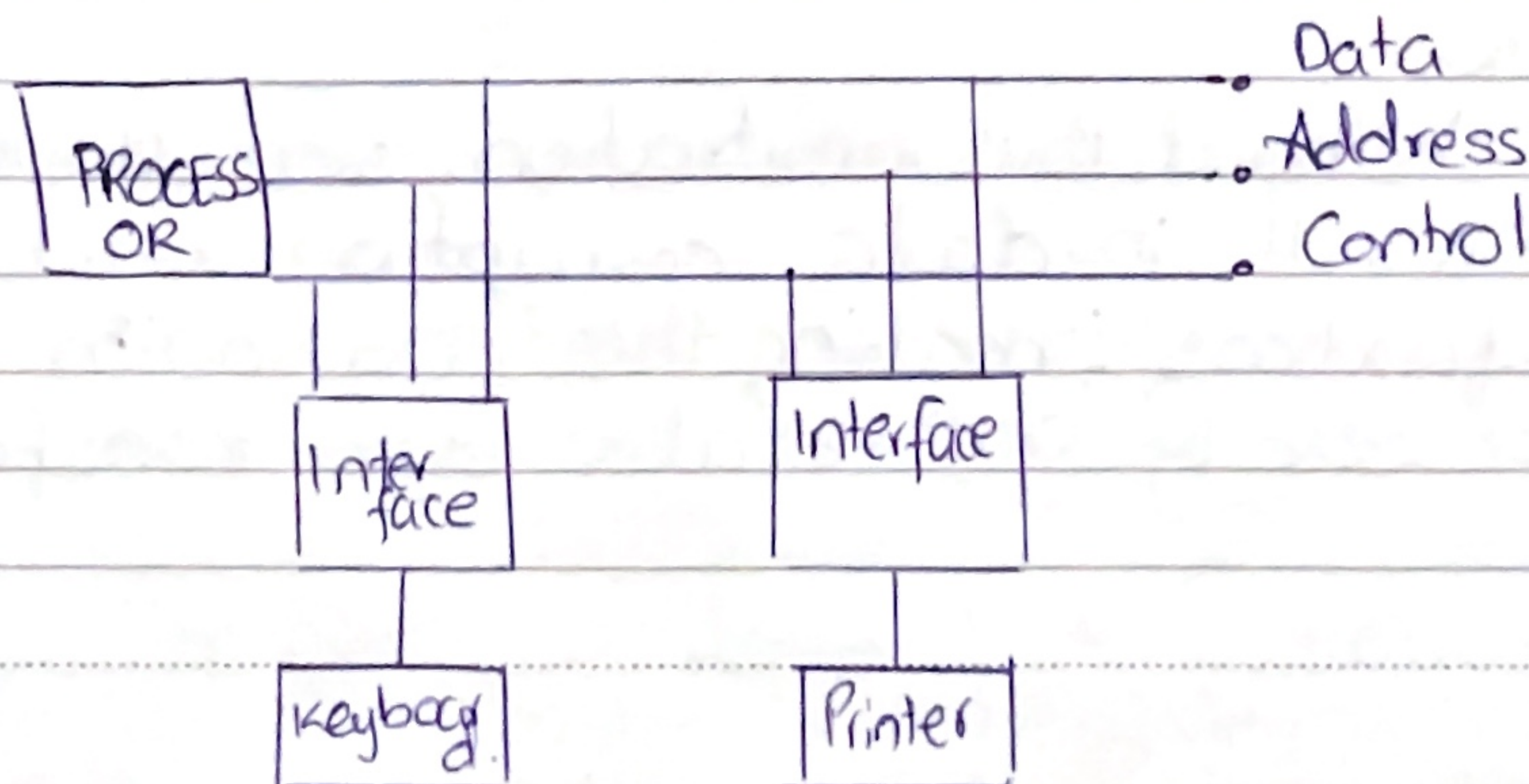
memory $\rightarrow$ register $\rightarrow$ ALU (perform operation) register $\rightarrow$ memory

BUSES:

Usually describes three types of Buses.

- (i) Address
- (ii) Control
- (iii) Data.

ADDRESS :- It is used to identify the correct I/O device among the no. of I/O devices; so CPU puts an address of a specific I/O device on the address line, all devices keep monitoring this address bus and decode it and if it matches then it activates control and data lines.



Sequence of usage
① → Address } buses
② → control
③ → Data

Date

Now after identification of the address of the right I/O device. Control Bus comes into action.

CONTROL BUS : After selecting the correct I/O device the CPU sends a functional code on the control line. The selected device (interface) reads that functions/functional code and execute it.

Example: I/O device command, control command status command etc.

DATA BUS :- In the final step depending on the operation either CPU will put data on the data line and device will store it or device will put data on the data line and CPU will store it.

Q. Describe BUS Arbitration.

This method is used to decide which device gets access to the common bus when multiple device requests it simultaneously, ensuring data integrity and system stability.

Conflict Resolution

Without Bus arbitration, simultaneous access could result in data corruption and system malfunctions, making this mechanism essential for orderly and reliable data transfer.

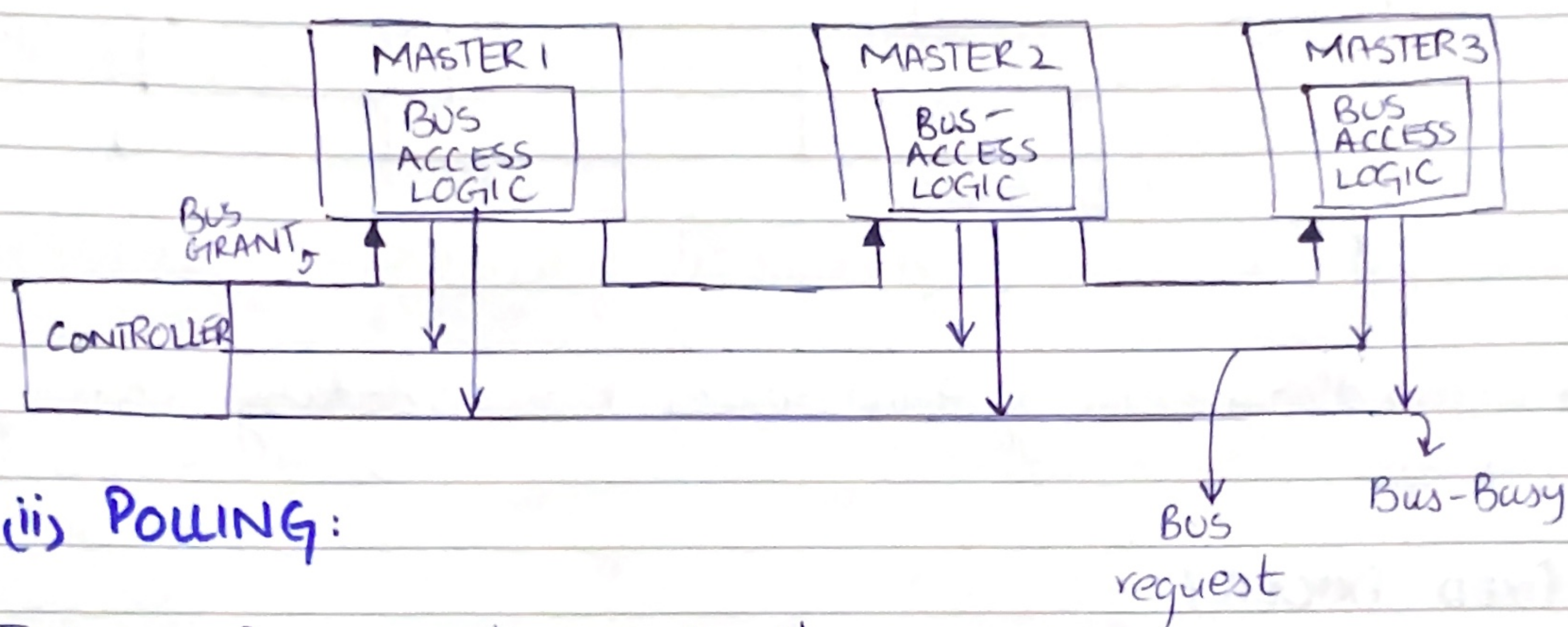
Date

Studying Methods of BUS ARBITRATION.

~ changeable Priority.

(i) **Daisy Chaining Method:** (Hardware positioning is imp! to change the priority)

It is simple and cheaper method where all the bus masters use the same line for making bus requests. The bus grant signal serially which propagates through each master until it encounters the first one that is requesting access to the bus; This master blocks the propagation of the bus signal, therefore any other requesting modules will not receive the grant signals and hence cannot access the bus.



(ii) **POLLING:**

It has 2 major steps to it.

→ Address Generation

In this method, a controller generates a unique address for a master device based on their priority. (meaning the order of these generation is based upon the device priority; higher priority device = early add gen)
The no. of address lines correlate with the no. of masters in the system. (if there are 8 devices to a system, it would need enough address lines too)
(devices) (for each device)

When the address of a particular master device is generated and

→ Sequence & Activation

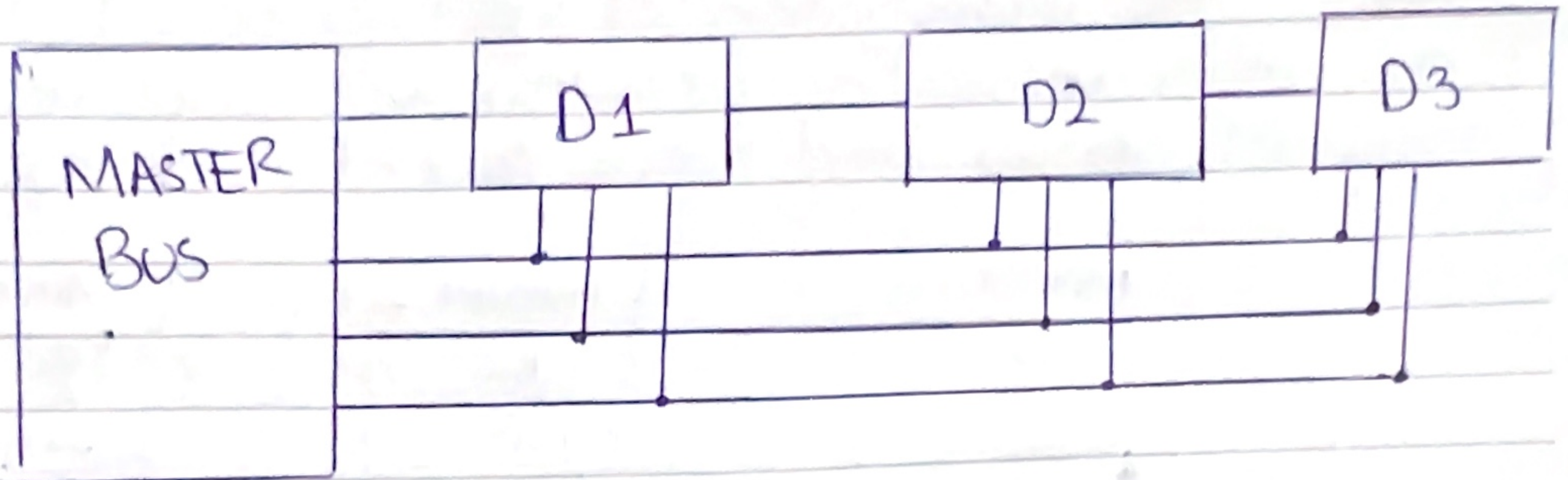
systematically.

in a fixed order

The controller cycles through generated addresses

When a master device recognizes its own address, it activates a 'busy' signal and gain access to the bus for data transfer.

After completing data transfer, the bus becomes free and the controller continues cycling through the address.



* we can change the priority of the devices to access data by changing the code.

~ FIXED PRIORITY

(iii) INDEPENDANT REQUEST METHOD.

In this method, each master has a separate pair of bus request and bus grant lines and each pair has a priority assigned to it.

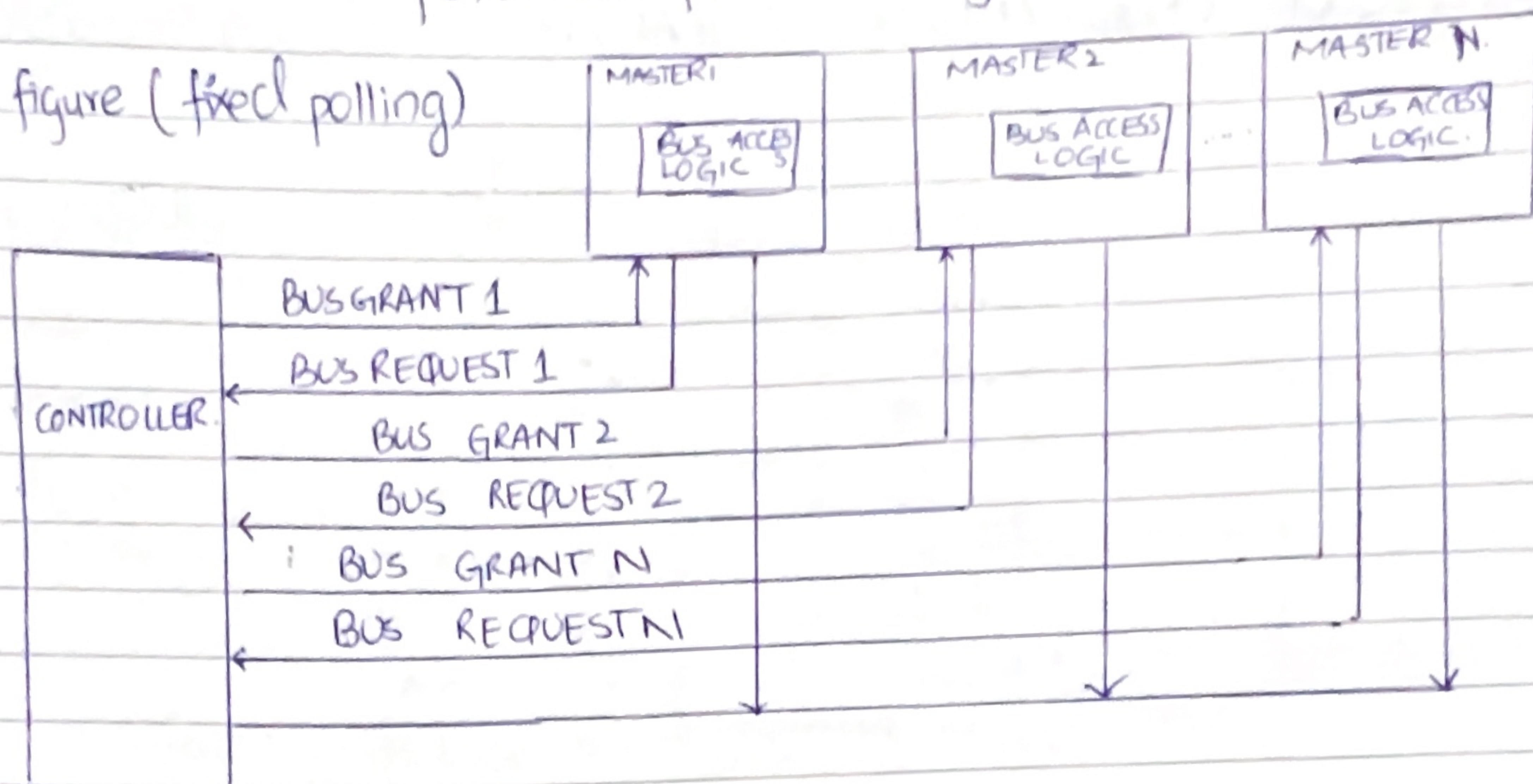
The built in priority decode within the controller selects the highest priority request and asserts the corresponding bus grant signal.

Registers

Date

Registers are high speed storage spaces in the CPU. The data processed by the CPU are fetched from the registers.

figure (fixed polling)



Q: Processor Organization.

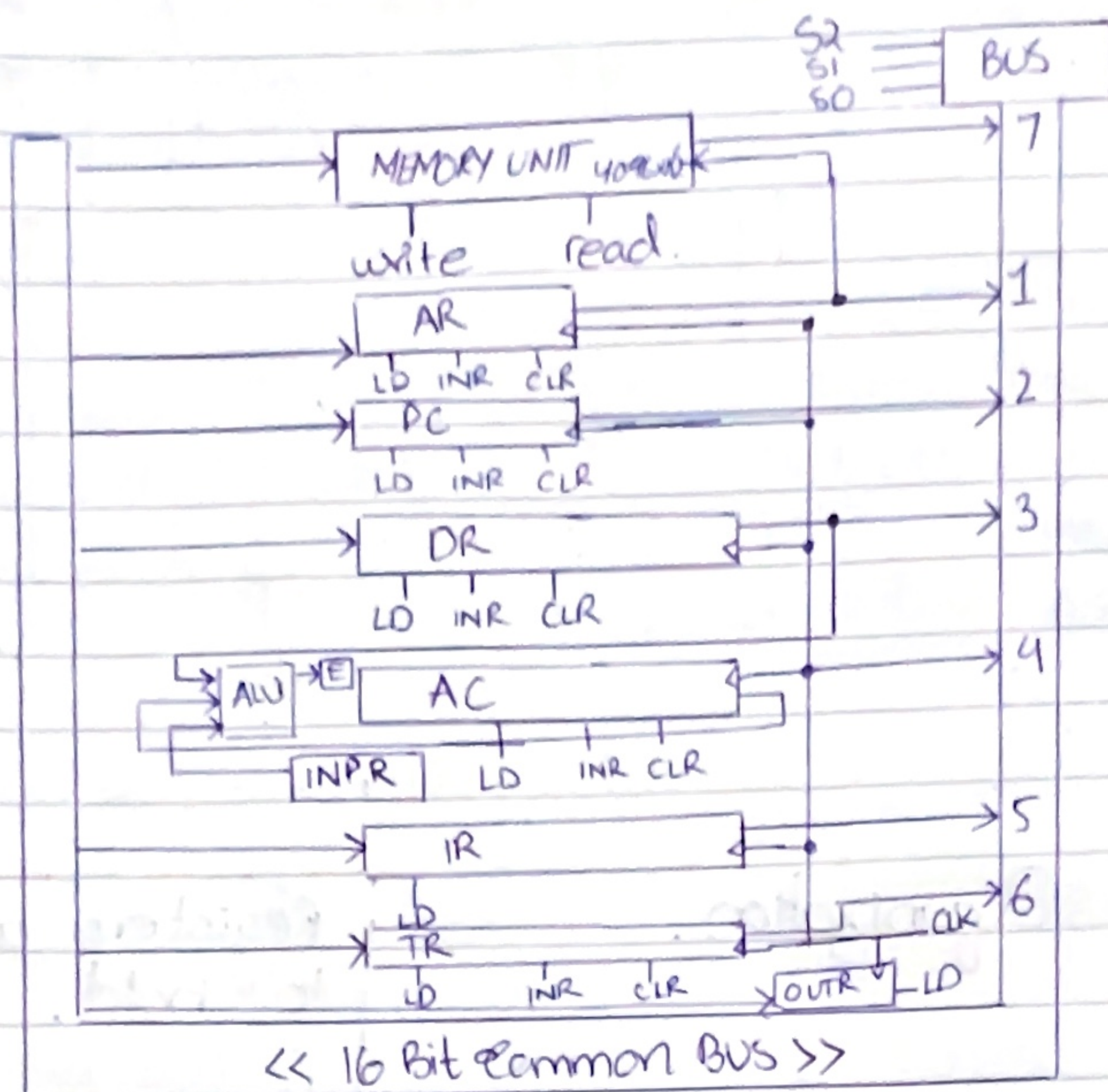
Consider the following diff types of registers first.

Registers are designed to hold.

- (i) Data being processed
- (ii) Instructions // executed
- (iii) immediate results or immediate calculations

Register symbol	No. of Bit	Register Name	funct.
DR	16	Data Register	holds memory operand.
AR	12	Address "	holds address for memory.
AC	16	Accumulator	Processor Register
✓✓ IR	16	Instruction Register	holds inst. code
✓✓ PC	12	Program Counter	addr. of inst. data.
TR	16	Temporary Reg	holds temp data.
INPR	8	Input Reg	holds input char
OUTR	8	Output Reg	holds output char.

PROCESSOR ORGANIZATION DIAGRAM:-



IMPORTANT REGISTERS:-

DR (Data Register):

A register which is used to store any data that is to be transferred to memory or which are fetched from memory.

Instruction Register (IR):

It is used to store instructions which we fetched from memory so that we analyse the instructions using a decoder and can understand what instructions are to be performed.

AC (ACCUMULATOR REGISTER)

The accumulator register is a general purpose processing register. Will hold the intermediate arithmetic and logic results of the operations performed in the ALU as ALU is not directly connected to memory.

* it can also load contents into itself through a specified memory location.

Date

Temporary Register (TR).

Used for holding temporary data during the processing.

Address Register (AR)

It has 12 bits since this width of memory address.

12 bits for the

i.e. $4096 \Rightarrow 2^{12}$ from the figure.

12 $\Rightarrow$ width of memory address
 $4096 \Rightarrow$ no. of unique memory locations.

* Address Register store the memory locations of instructions or data; helping to be located from memory.

PROGRAM COUNTER (PC):-

It is also 12 bits and it holds the address of the next instruction to be read from the memory after the current instruction is executed.

The 'PC' goes through a counting sequence and causes the computer to read sequentially (from the already stored instructions in memory).

INPR. (INPUT REGISTER):

receives an 8-bit character from an input device and pass it to ALU then accumulator and then to the memory.

OUTR. (OUTPUT REGISTER)

holds an 8-bit character for an output device, character printer etc.

NOTE:-

- The output of the 7 registers and memory are connected to the common bus.

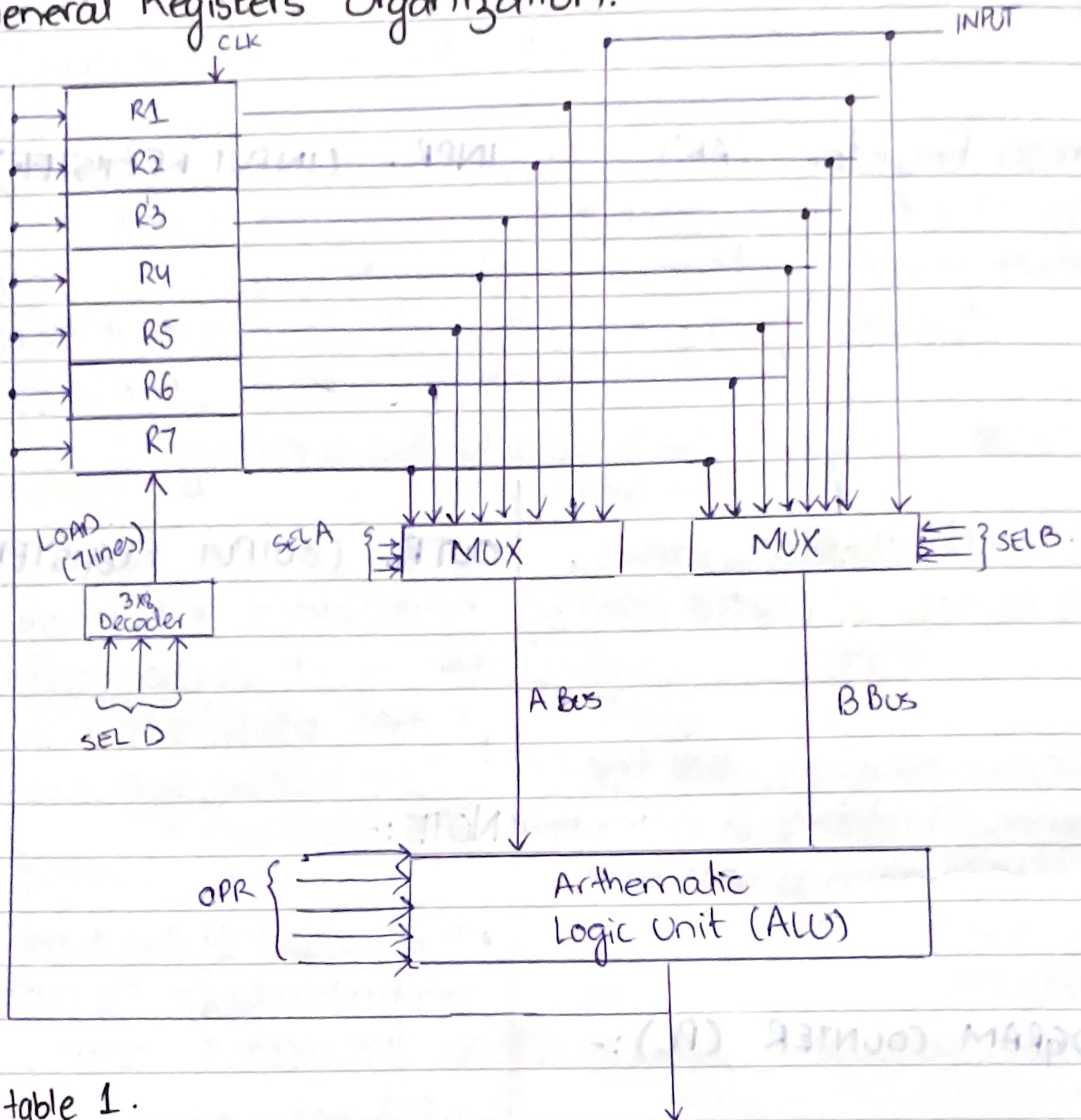
- The specific output that is selected for the bus line at any given time is determined from the binary value of the 3 selection lines i.e. S_0, S_1, S_2

Date

- The number along each output shows the decimal equivalent of the required Binary selection.

Chapter no.8

General Registers Organization.



Imp table 1.

Binary Code	SELA	SELB	SELD.	Output
000	Input	Input	None.	
001	R1	R1	R1	
010	R2	R2	R2	
011	R3	R3	R3	
100	R4	R4	R4	
101	R5	R5	R5	
110	R6	R6	R6	
111	R7	R7	R7	

k- Register of n bits

MUX shape = (1×1)

MUX no. = n
no. of selection line = $2^b = k$

Date

$b = \lceil \log_2 k \rceil$

Important Table 2.

OPR Select	Operations.	Symbol.
00000	Transfer A	TSFA
00001	Increment A	INCA
00010	Add A+B	ADD
00101	Subtract A-B	SUB
00110	Decrement A	DECA
01000	AND A and B	AND
01010	OR A and B	OR
01100	XOR A and B	XOR
01110	Complement A	COMP
10000	Shift-Right A	SHRA
11000	Shift left A	SHLA

Example:- Performing microoperation $R1 \leftarrow R2 + R3 + 1$

SELA $\Rightarrow$ 010 (for R2) $\rightarrow$ Control code.

SELB $\Rightarrow$ 011 (for R3) $\rightarrow$ $\Rightarrow$ 010 011 001 00101

ALW $\Rightarrow$ 00101 (for SUBTRACTION) $\rightarrow$

SEL0 $\Rightarrow$ 001 (for loading) $\rightarrow$
back

EMPTY REGISTER.

EMPTY REGISTER

DATA REGISTER

Stack Organization.

It is an ordered list where addition of new data item and deletion of already existing data item is done only from one end known as the top of the stack.

The element which is added in last will be first to be removed and the element which is inserted first will be removed at the last, so it is called last in first out (LIFO) or first in Last out (FILO) type of list.

Most frequently accessible element in the stack is the topmost element, whereas the least accessible element is the bottom of the stack.

REGISTER STACK.

STACK POINTER REGISTER :- (SP)

It contains a value in binary each of 6 Bits, which is the address of the top of the stack. Here, the stack pointer (SP) contains 6 Bits and SP cannot contain a value greater than 111111 i.e 63.

FULL REGISTER

It can store 1 Bit of information, set to 1 when stack is full.

EMPTY REGISTER.

It can store 1 Bit of information, set to 1 when the stack is empty.

DATA REGISTER.

It holds the data, to be written into or to be read from the stack.

Date



Address width = 16 Bits

∴ Size of SP $\Rightarrow$ 16 Bits

Process Breakdown:-

Push the operands in stack (until)
pop them when an operator arrives (and)
push the result back to the stack
(This continues until the very final step after which the final result is popped and stored in DR to be finally saved in the Memory).

MEMORY STACK.

Memory stack operates last in first out as well, where the most recent added element is first to be removed

A special transfer called the stack Pointer (SP) points to the top of the stack and it can contain binary values upto a limit, often determined by the architecture such as 6 Bit ... equating to 63 in decimal.

... $\Rightarrow$ every thing else similar just using memory instead of Registers

Chapter no. 5

ADDRESSING MODE:-

It specifies the different ways possible in which reference to the operand can be made.

Effective Addl It is the final address of the location where the operand is stored.

Calculation of effective address can be done in two ways.

- ▷ Non-computable addressing.
- ▷ Computable address (which involves arithmetics).

'CRITERIA FOR ADDRESSING MODES'

- It should be fast.
- The length of the instruction must be small.
- They should support pointers
- They should support looping constructs, indexing of data
- structure
- Program relocation.

IMMEDIATE MODE

It means that the operand itself is the part of the instruction (this instruction code which usually contained op code and memory address, now actually has the operand in it).

ADD 3

Means add 3 to the accumulator.

OP CODE	OPERAND
---------	---------

Date

ADVANTAGE

Can be used for constants, where values are already known.
Extremely fast, no memory reference is required.

DISADVANTAGE

Cannot be used with variables whose values are unknown at the time of program writing.

Cannot be used for large constant whose values cannot be stored in the 8-bit part instructions.

APPLICATION

Mostly used when required data is directly moved to required register or memory.

DIRECT MODE ADDRESSING (Absolute Address Mode).

It means instruction contain address of the memory location where data is present (Effective Address).

* Here only one memory reference operation is required to access the data.

ADVANTAGE

- With variable whose values are unknown direct addressing is the simplest form.
- No restriction of range of data, and largest value which system can hold can be used in this mode.
- Can be used to access global variable whose address is known at compile time.

Date

DISADVANTAGE

- Relatively slow compare to immediate mode.
- No. of variable used are limited.
- In large calculation it will fail.

INDIRECT MODE ADDRESSING

Here in the instruction we store the address where the (address of the variable) effective address is stored using which we can access the actual data.

Here two references are required.

- 1st Reference to get the effective address and
- 2nd Reference to access the data.

ADVANTAGES

- No limitation on ^{→ or size of} no. of variables.
- Implementation of pointer are feasible and relatively secure

DISADVANTAGES

- Relatively slow as memory must be referred more than one time.

Date

IMPLIED MODE ADDRESSING

Implied Addressing mode, the operands are specified implicitly in the definition of the instruction.
↳ $\frac{A}{\text{Accumulator}}$

All the instructions which reference registers that use an accumulator are implied mode instructions.

Zero address instructions in a stack organized computer are also implied mode instructions.
↳ this is known as stack addressing mode.

Example: Increment Accumulator.

Complement Accumulator.

REGISTER MODE ADDRESSING:- (it can also have an indirect mode)

It means the operand that we are trying to locate is now present in the registers of the CPU instead of the memory. In the instruction we will thus have an opcode, register no.

ADVANTAGE

- (i) Extremely fast because register access time is less than cache access time.
- (ii) Because no. of registers is less, so bits required to specify a register is also less.

DISADVANTAGE

As no. of registers is less only a few instructions can be used.

Date

BASE REGISTER (OFFSET) MODE

In multiprogramming environment, the location of the process in the memory keeps on changing from one place to another.

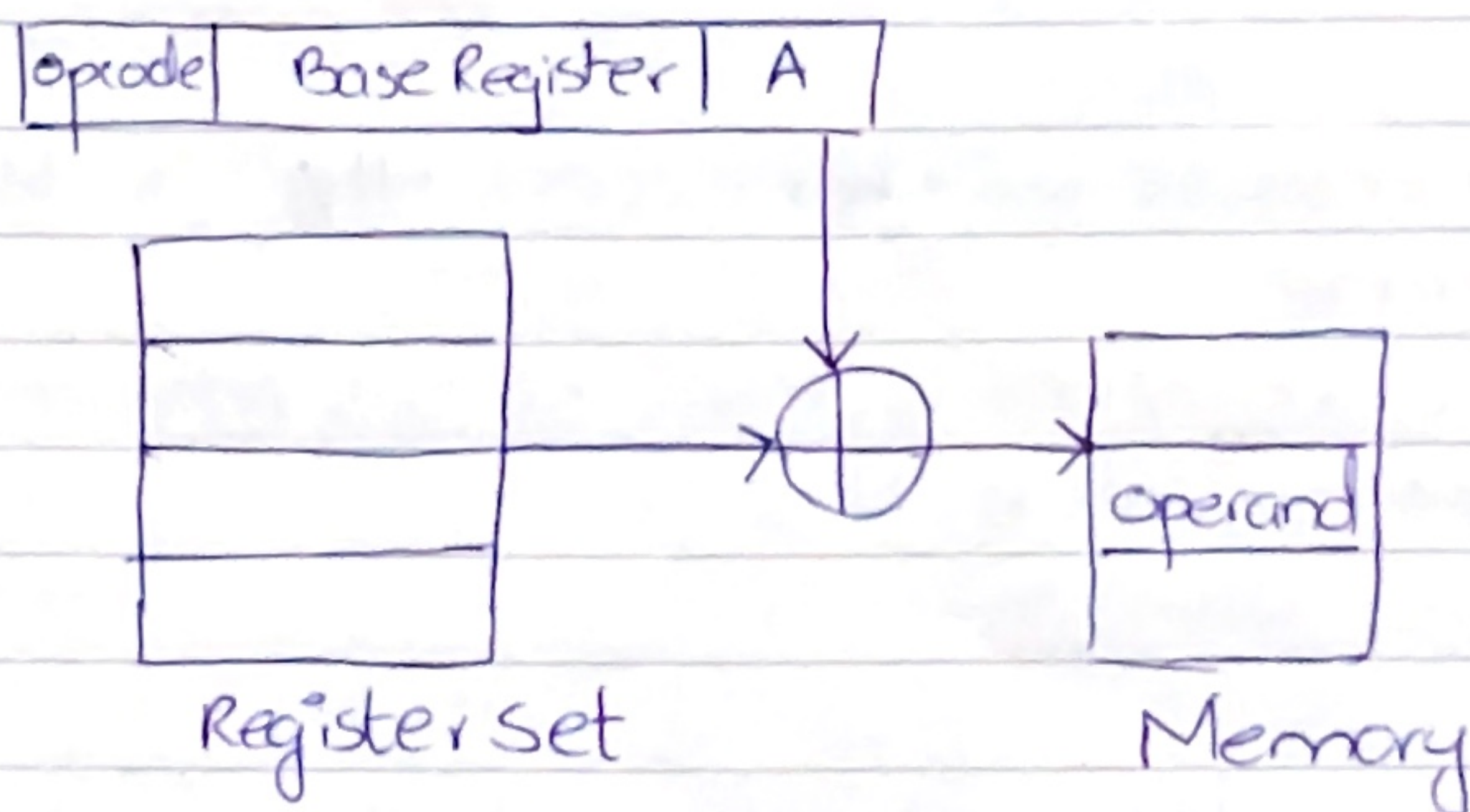
But if we are using direct Address in the process, then it will create a problem. So, to solve this problem we try to save the starting of the program address in a register called Base register.

Then instead of giving the direct address branch we give off in the instruction.

$$\text{effective address} = \text{Base address} + \text{offset (instruction)}$$

ADVANTAGE

Even if we are trying to shift process in the memory, we only need to change the contents of the base register.



Single operand instruction/structure uses an implied accumulator register for all data manipulations. Implied means that CPU has one operand in the accumulator already and other

INSTRUCTION FORMAT. (pt.1)

INSTRUCTION_x

A statement that tells the computer what to do.

is taken through effective address of instru.

INSTRUCTION_x FORMAT

The way in which instruction is given is called as the instruction format.

A computer has a variety of instruction formats.

There are two common parts of the instruction format that are as follows:

Operand Code

Address of a Operand.

The operation code / operand code specifies the operation to be performed by the computer such as ADD, SUB and MOV etc

The address of operand refers to a location in where the value is stored

GENERAL INSTRUCTION FORMAT

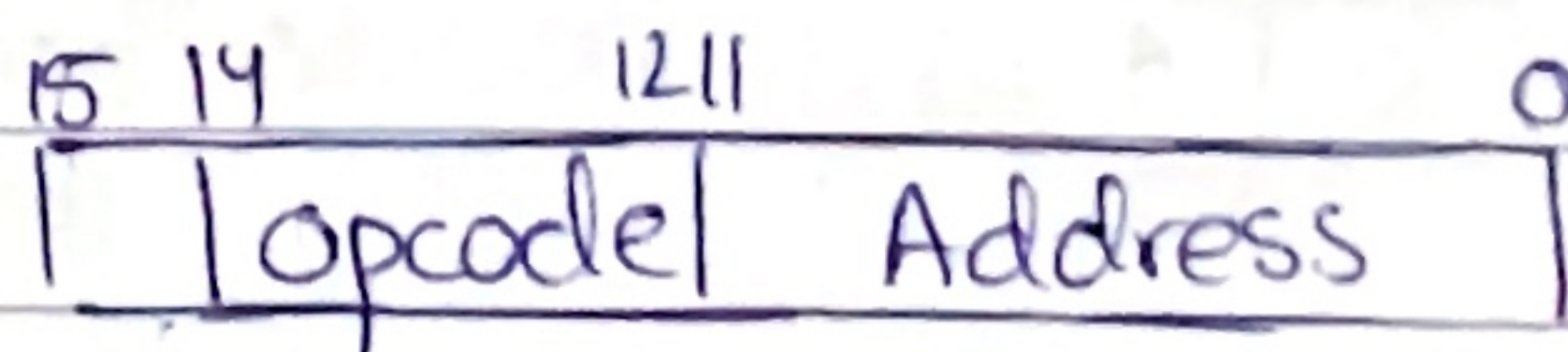
(single operand structure)

The architecture we are studying consist of; 16 Bit Instructions

12 Bit Address Operand.

3 Bit Opcode

1 Bit Address mode Bit.

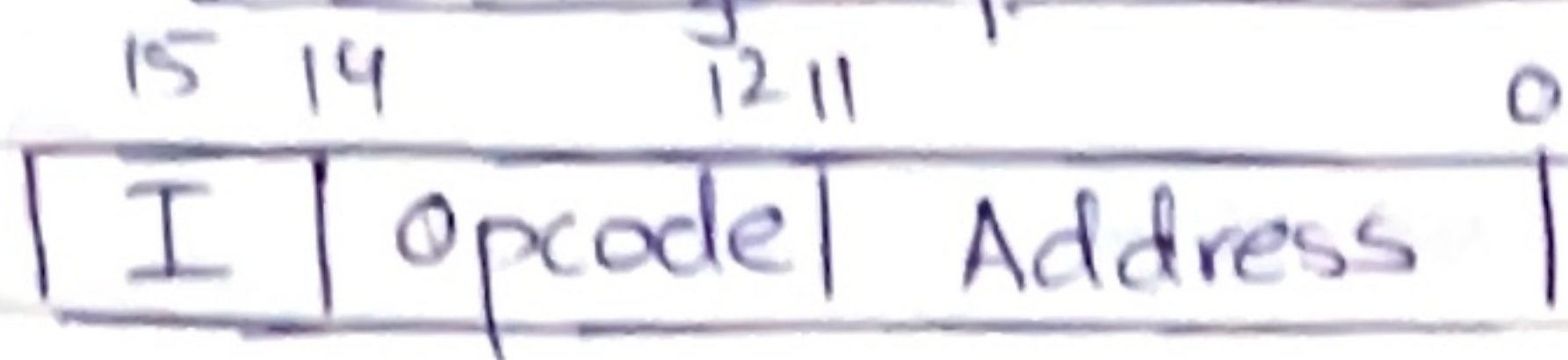


Now further there are different instruction types as per this instruction format (each having own structural preferences).

→ Opcode ranges from 000-111

Date

(a) Memory Reference Instructions.



→ These instructions involve reading and writing to memory.

I Bit: it determines the addressing mode.

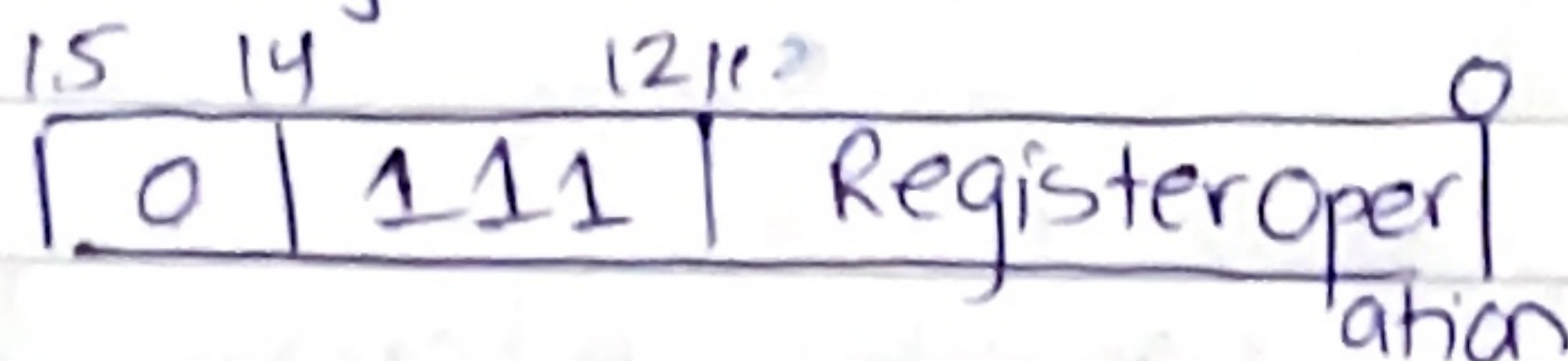
I=0 means direct Addressing Mode
(i.e. Address field contains the address of memory location).
↓
target

I=1 Means indirect Addressing Mode.
(i.e. Address field points to a memory location that actually contains address of target).

opcode (3 bit: 14-12 Bit) This determines the operation to be performed.

* Address (bits 11-0): In this instruction type it specifies the memory location (that will contain the target).

(b) Register-Reference Instruction



These instructions operate directly on registers
→ CPU.

Since

I=0

& OP=111

(12 Bits field (11-0) specifies the register.
operation.

→ Since they don't require an address, they involve only registers
→ These instructions are executed in a single cycle; not requiring memory access.

Examples of Register Operations.

- (i) Clear Accumulator.
- (ii) Complement and accumulator.
- (iii) Shift operations.
- (iv) Increment / Decrement.

↓ Note

In this case the operand or value on which operation is to be performed should already be present in register before execution.

(c) INPUT/OUTPUT INSTRUCTION

$I = 1$

Opcode = 111

These instructions are used to communicate between CPU and INPUT/OUTPUT Devices.

12 Bit field (11-0) that specify the I/O operations.

↓
e.g.

- Input
- Output
- Check device status etc.

'UNDERSTANDING THROUGH EXAMPLES'

EXECUTION CYCLE OF MEMORY REFERENCE INSTRUCTIONS.

every instruction follows a sequence of operations.

* consider (ADD 810) add operation

Step no. 1 FETCH CYCLE

- ① the program counter provides the address of the instruction to MAR.
- ② the instruction is fetched from the memory
- ③ the fetched (Memory's) instruction is then stored in the IR.
- ④ PC is incremented to point to the address of the next inst.

Date

Step no. 2

DECODE CYCLE.

- ① The CPU decodes the instruction.
- ② Here it finds an OPCODE to know the operation, the addressing mode and operand's address to access the operand.

STEP NO. 3

EXECUTE CYCLE.

- ① The address field is sent to MAR.
- ② Memory returns the value stored / operand from the address.
- ③ The value is temporarily stored at MDR. from there it is added to the Accumulator.

'INSTRUCTION SET COMPLETENESS'

Instruction set completeness refers to the property of a computer that ensures that it has all the necessary instructions to perform any computable function.

A complete instruction set allows user to create machine language programs capable of solving a wide variety of problems.

A minimal yet complete instruction set generally includes.

* Efficient Instruction will allow subtraction in 1 step.
(however usually subtraction occurs in 2's complement).
* NAND can do any instruction possible.

(i) FUNCTIONAL INSTRUCTIONS:-

These perform arithmetic, logic and shift operations.

ADD

CMA (Complement accumulator).

INC

CIL

AND

CLA (Clear Accumulator).

(ii) TRANSFER INSTRUCTIONS:-

These deal with moving data between Registers and memory.

LDA (Load Accumulator)

STA (Store Accumulator)

(iii) CONTROL INSTRUCTIONS:-

Used for program sequencing and control.

BUN (Branch Unconditionally) ^{out a condition} Jump to a specific address with

BSA (Branch and Save Return Address)

ISZ (Increment and Skip if Zero)

↳ Increments a register and skips the next instruction if result is zero.

used for subroutine calls; saves the return address before branching.

(iv) INPUT/OUTPUT INSTRUCTIONS:-

These handle the data exchange b/w CPU and external devices

IMP
OUT

Date

'CONTROL UNIT'

It is the Brain of Digital System/Computer. Control unit of a processor translates machine instructions into control signals; for the micro-operations that implement them.

Control units are implemented in one of the 2 ways.

(i) Hardware Control (fast and efficient) / optimized

Control unit is made up of a sequence of circuits (sequential and combinational) to generate the control signals.

If control logic changes we need to change all components.

(ii) Micro programmed Control less optimized.

Control memory on the processor contains microprograms that activate the necessary control signals. changes in the control memory.

To change the control logic only make $\uparrow$ hardwired Control Unit

* Morris Mano's Basic computer consist of $\uparrow$ Basic Computer

TIMING CIRCUIT

In order to perform an instruction we need to perform a number of micro-operation and these micro-operations must be timed.

* we should distinguish one clock pulse (performing one micro operation) from another during the execution of instruction.

Sequence Controller/Counter followed by decoder is used to generate time signals.